

leanmend: salvaging broken Lean 4 projects, and the verified development built around it

The *request-project* authors*

26 August 2026

Abstract

A Lean 4 package either builds or it does not. When it does not — a stale toolchain, a miscapitalised import, a dependency that moved, one proof that broke after a Mathlib bump — the compiler is a poor diagnostic instrument: an unresolved import is reported at line 1 and elaboration of the module stops there, so nothing downstream is ever seen, and an import cycle surfaces only as a recursion failure. Work that is perfectly sound elsewhere in the package becomes invisible and unusable.

We present **leanmend**, a dependency-free Lean 4 tool that parses, splits, diagnoses, and mechanically repairs broken Lean 4 packages, and — the capability we consider the most useful — extracts *minimal self-contained slices* of a package that compile on their own, one per declaration, even when the package as a whole does not compile. **leanmend** is purely textual where it has to be (so it works on files that do not parse) and calls the real compiler where it must (so every “this compiles” is an actual compiler run). We validate the block scanner against 200 randomly chosen Mathlib files (54,511 lines, 10,363 blocks) with an independently implemented checker, report a 69-assertion end-to-end suite, and measure package-scale salvage: all 162 named declarations of **leanmend**’s own 12-module package are recovered as slices that individually compile, with a median slice of 3 declarations and 63.5 lines.

Around the tool sits a **sorry-free** Lean library of 11,289 lines and 429 theorems in five subsystems — a verified catalog builder, a self-hosting circuit development, a complexity lattice that measures the tool’s own slice space, an architectural layer, and a formally verified zero-knowledge proof-of-proof construction. Their headline theorems’ axiom closures are pinned inside the build itself, so the audit cannot silently go stale. We are explicit throughout about what is *not* claimed.

1 Introduction

Large Lean 4 developments break in ways that are mundane and expensive. A project written on a case-insensitive filesystem imports `MathLib.Order.Basic` and fails on Linux. A Mathlib bump renames a lemma and a single `linarith` call stops closing its goal. A manifest goes out of date and a dependency is not where the lakefile says it is. In every one of these cases the package stops building, and the rest of its contents — often thousands of lines that are still perfectly correct — become inaccessible: they cannot be compiled, cannot be checked, and cannot be reused, because Lean’s unit of work is the module and a module’s unit of work is the whole import prefix.

The compiler is not designed to help here, and in one specific respect it actively cannot. When `import Foo.Bar` does not resolve, Lean reports the error at line 1 — irrespective of which import line is actually at fault — and elaborates nothing else in the file. A user is

*The repository was developed with Aristotle (Harmonic). Every number reported in this paper was produced by a command run against the commit this paper is committed with; §9 lists the commands.

told that *something* in the header is wrong and given no information about the 2,000 lines beneath it. Import cycles are worse: they appear as deep-recursion errors naming an arbitrary participant.

This paper describes `leanmend`, a tool built for exactly that situation, and the verified Lean development that grew around it. Our contributions:

1. **A robust textual block model of Lean sources** (§3.1) that splits a file into top-level blocks with line ranges and name-level dependencies without parsing, elaborating, or building anything — validated on 200 Mathlib files against an independently written checker (§4.1).
2. **Cross-module slicing and package-scale salvage** (§3.2, §3.3): `extract --inline` resolves names the way Lean does — own module first, then transitively imported modules — and emits one file that inlines every internal dependency and imports none of the package’s own modules, so it compiles even when the package does not. `harvest` does that for every declaration in the package and compiles each result, producing a manifest of exactly what could and could not be salvaged.
3. **Import-graph diagnosis without compilation** (§3.4): missing modules with edit-distance suggestions, imports that differ from a real module only in case, duplicated and unused imports, cycles printed as cycles, and a build order — all read off the `import` lines, hence available on a package that cannot be built at all, or has never been fetched.
4. **A repair policy that never invents mathematics** (§3.5): import spellings are corrected, failing proofs become explicit `sorry`s that the tool can find again, unsalvageable declarations are commented out line by line, and the original file is always preserved.
5. **A sorry-free verified library** of 11,289 lines and 429 theorems (§5) in five subsystems, including a catalog builder whose invariants are theorems rather than tests, and a complexity lattice that takes `leanmend`’s own harvest output as its object of study — the tool measured by the library it ships with.
6. **A machine-checked axiom audit** (§5.6): the axiom closure of every headline theorem cited in this paper is pinned inside the build with `#guard_msgs`, so the build fails if a cited theorem ever acquires a non-standard axiom.

2 Background

Lean 4 and Lake. A Lean 4 package is a set of modules compiled by Lake into `.olean` files. Compilation is transitive and all-or-nothing per module: a module is elaborated only if every module it imports has been built. Diagnostics are available in structured form via `lean --json`, and `lake env lean` runs the compiler inside the package’s environment (search paths, dependencies). `leanmend` drives both.

The kernel. Lean’s trusted computing base is its kernel. `lean4lean` [1] is an independent implementation of that kernel in Lean itself, usable to re-check a built package. `leanmend` integrates it as an *optional* back end (`leanmend kernel`), fetched and built on demand by `vendor/fetch-lean4lean.sh` against its own pinned toolchain, since kernel code is tied to the Lean version it reads `.olean` files for.

command	what it does
<code>doctor [DIR]</code>	toolchain, lakefile, dependencies, manifest, import graph, build
<code>blocks FILE</code>	top-level declarations with line ranges and dependencies
<code>diagnose FILE...</code>	compile and attribute every error and warning to a declaration
<code>diagnose --project</code>	build the package, then diagnose exactly the modules that failed
<code>split FILE --out DIR</code>	one file per declaration (<code>--standalone</code> : each compilable)
<code>extract --decl N</code>	the minimal self-contained slice defining <code>N</code>
<code>extract --decl N --inline</code>	... slicing across modules, importing none of the package
<code>harvest --out DIR</code>	one compiled slice per declaration of the package, plus a manifest
<code>graph [DIR] [--fix]</code>	the package's own import graph, without compiling anything
<code>sorries FILE... / --project</code>	declarations still containing <code>sorry</code> / <code>admit</code>
<code>repair FILE / --project</code>	mechanical repair until the file or package builds
<code>kernel [MOD]</code>	re-check the built package with an external <code>lean4lean</code>

Table 1: The `leanmend` command surface.

Design constraint: no dependencies. `leanmend` is written in Lean 4 and depends on Lean core only — no Mathlib, no Batteries. It therefore builds cold in under a minute and can be pointed at any package without being added to that package's lakefile or perturbing its dependency graph. This matters precisely in the broken-project case, where adding a dependency is the last thing one wants to do.

3 Design

`leanmend` is 2,572 lines across 12 modules. Table 1 lists the commands; each has a `--json` mode for scripting, and the common options `--root DIR`, `--timeout N` and `--no-lake`.

3.1 The block scanner

Everything rests on a comment- and string-aware line scanner (`LeanMend/Scanner.lean`) that partitions a file into top-level *blocks*. It is deliberately textual: the files we care about are exactly the ones that do not parse, do not elaborate, or import modules that were never built, so no syntactic or semantic service of the compiler is available.

A block boundary is a line that starts in column 0 with a top-level command keyword, outside comments and strings. Three refinements matter in practice:

- Doc comments and attributes stay attached to the declaration they decorate; a block starts at the doc comment, not at the `theorem` keyword.
- `mutual ... end` groups are one block, since their members cannot be separated.
- `open X in def f` is classified as the declaration `f`, not as a context command. This one was a real bug found by running the tool on its own source: classified as context, the prefix was replicated into every slice, dragging dangling references along and breaking slices that were otherwise valid. Commands whose body merely contains the token `in` (`elab`, `macro`, ...) are left alone.

Blocks are then classified as *context* (module doc, imports, `open`, `namespace`, `variable`, `set_option`, notation) or *declarations*, and a declaration's *dependencies* are the names it mentions that another block declares. Resolving those names is subtler than it looks: `!` is a name character in Lean, so `!xs.contains y` is a use of `xs`; and dot notation (`st.inCode`) must be resolved through dotted prefixes and tails, or slices silently come out incomplete.

3.2 Slicing

The slice for a declaration d is the file header, all context commands, and the transitive closure of d 's dependencies in block order. `--verify` compiles the result with the real compiler; `--minimize` then delta-debugg it, recompiling the slice with each dependency removed in turn and dropping the ones that turn out not to be needed. Minimisation is what makes the “purely textual dependencies” approximation safe: the textual pass over-approximates (a name mentioned in a comment-like position, or shadowed locally, is kept), and the compiler prunes the over-approximation.

Single-file slicing does not help when the package as a whole is broken, because the slice still carries the file's imports — including imports of the package's own, unbuildable modules. The `--inline` mode (`LeanMend/Project.lean`) removes that dependency: every module of the package is scanned; each mentioned name is resolved in its own module first, then only in the modules that module transitively imports, as Lean itself resolves; the contributing modules are emitted in import order; the package's own imports are *dropped*, their declarations having been inlined, and external imports (Mathlib, say) are kept. The emitted file therefore depends on nothing inside the package.

An `open N` line that survives into a slice that inlined nothing into namespace `N` is rejected by Lean, so such names are dropped from the `open` line and the line disappears when nothing is left. Complex forms (`open ... (...)`, `scoped`, `renaming`, `hiding, ... in`) are left untouched rather than mangled.

3.3 Harvesting a whole package

`harvest` is `extract --inline --verify` applied to every declaration of the package in turn: each slice is compiled on its own, the ones that compile are written to `--out DIR`, and `manifest.json` records for every declaration its module, slice size, whether it compiled, and the errors when it did not. `--jobs N` compiles N slices concurrently; the output is independent of N , and the test suite asserts that a concurrent run produces exactly the same slices as a serial one. `--minimize` delta-debugg each slice (slow), `--keep-failing` writes the failures too, and `--decl / --limit` restrict the work.

This is the package-scale form of the original question: *extract the parts of a broken project that still run, and build them*. A declaration is salvaged even when its own module does not compile, because the module's imports have been inlined away; it is lost only when the declaration's own transitive dependencies are themselves broken.

3.4 Import graph without compilation

`graph` reads `import` lines and classifies each against the package's own modules as *internal*, *miscased* (matches a real module up to case — the classic macOS-to-Linux breakage), *missing* (a name in the package's root namespace with no module behind it) or *external*. It reports missing modules with the nearest existing module by edit distance, cycles found by depth-first search and printed as cycles, duplicated imports, internal imports from which the importing module mentions nothing, and a topological build order. It compiles nothing, so it answers on a package that cannot be built at all. Listing 1 shows the output on the test fixture.

Listing 1: `leanmend graph test/graphpkg` (exit code 1). Terminal glyphs are rendered here as `[x]`/`[!]`.

```
/workspace/request-project/leanmend/test/graphpkg: 6 module(s), 8 internal import(s)
[x] import cycle: G.A -> G.B -> G.A
[x] G.C imports G.Halper: no such module (did you mean G.Helper?)
[!] G.B imports G.A but uses nothing from it
[!] G.C imports G.Helper more than once
```

```
[!] G.Lonely imports G.Helper but uses nothing from it
build order: G.B, G.A, G.Helper, G.C, G.Lonely, G
2 broken import(s)/cycle(s), 3 warning(s)
```

`graph --fix` rewrites only what the analysis is certain about — a miscapitalised import gets the spelling the module actually has, a repeated import is deleted — keeping each rewritten file’s original as `FILE.orig`. A missing module is never guessed at, and an unused import is never removed: instances, notation, attributes and `simp` lemmas are invisible to a textual analysis. For the same reason an unused import is a warning and never an error, and aggregator modules (which only re-export) are excluded from that check.

Running the check on the tool’s own package immediately found a genuine redundancy — `LeanMend/Doctor.lean` importing `LeanMend.Repair` and using nothing from it — which was removed.

3.5 Diagnosis, repair, and doctor

`diagnose` runs `lake env lean --json` and attributes each structured diagnostic to the block whose line range contains it, so the output is a list of *declarations* that are broken and why, rather than a list of line numbers. `--project` first parses `lake build` output for the modules that failed and diagnoses exactly those.

`repair` is a fixed loop of mechanical steps: correct module spellings; comment out imports the compiler cannot resolve — identified *by the module name inside the error message*, since Lean reports every such error at line 1 whatever line the offending import is on; then repeatedly compile, take the first failing block, replace its proof with `sorry`, and, if it still fails, comment it out line by line. The policy is deliberate: **a repair never invents mathematics and never destroys source text**. A stubbed proof is an explicit `sorry` that `leanmend` sorries will find again, a removed declaration remains in the file as comments, and the original is saved as `FILE.orig`.

The line-1 subtlety was another bug found in use: the first version commented out the *first* import whenever an import failed, which deleted a perfectly good line and left the real offender in place.

`doctor` aggregates the package-level checks — `lakefile` present, `lean-toolchain` against the active Lean version, each dependency’s presence, revision and built state against `lake-manifest.json`, lake’s own “manifest out of date” warning, the import graph of §3.4, and the build itself with the failing modules named (Listing 2).

Listing 2: `leanmend doctor test/broken` on the deliberately broken fixture.

```
leanmend doctor: /workspace/request-project/leanmend/test/broken
[ok] lakefile: lakefile.toml
[ok] toolchain: leanprover/lean4:v4.28.0 (Lean (version 4.28.0, ...))
[ok] dependencies: no git dependencies in lake-manifest.json
[ok] sources: 3 .lean files (excluding dot-directories)
[x] imports: Broken.Basic imports broken.Helper: miscapitalised, the module is Broken.
    Helper
[x] lake build: failed in modules: Broken.Basic
problems found
```

One resolution detail deserves mention because it changes what the tool can be pointed at: a file inside `<ws>/lake/packages/<pkg>` is resolved to the enclosing workspace `<ws>`, not to the vendored package, because only the workspace has an environment in which that package’s own imports resolve. This is what makes `diagnose` and `extract` work out-of-the-box on a *dependency’s* sources. Symmetrically, source enumeration skips any subdirectory carrying its own lakefile — a test fixture, an example, a vendored checkout is a separate package, not part of this one.

section	#	section	#	section	#
blocks	5	sorries	2	repair <code>--project</code>	2
doctor	6	extract	9	repair (imports)	3
diagnose	1	split	2	harvest	14
repair	5	kernel	4	graph	16
total 69, all passing					

Table 2: End-to-end assertions by section.

4 Evaluation

4.1 Scanner validation on Mathlib

The block model is the foundation of everything else, so we validate it against real, large-scale Lean source with a checker written independently of the tool (`leanmend/test/scanner_validation.py`): a separate comment- and string-aware pass in Python re-derives the facts the scanner is supposed to guarantee and checks the scanner’s JSON output against them. Three properties are checked per file:

- P1 (partition)** the blocks are ordered, pairwise disjoint, and every non-blank line of the file lies in exactly one block;
- P2 (boundaries are real)** no block starts on a line that the independent pass finds to be inside a block comment or a string literal;
- P3 (declarations are separated)** every column-0 declaration keyword found by the independent pass lies inside some block, and two of them share a block only when they are inside the same `mutual ... end` group.

On a seeded random sample of 200 Mathlib files — 54,511 lines, 10,363 blocks — all three properties hold for every file: *0 files with problems*. The sample is reproducible (`--seed`, default 20260826).

4.2 End-to-end suite

`leanmend/test/run_tests.sh` is 289 lines of end-to-end assertions run against real fixtures — a deliberately broken package and a package with a deliberately broken import graph — with the compiler actually invoked. All **69 assertions pass** (exit code 0), spread over the sections shown in Table 2. Every documented command is covered by at least one assertion, and several assertions encode the guarantees we advertise rather than mere outputs: that `--fix` leaves a missing module and an unused import alone, that a harvest with `--jobs 4` yields exactly the slices a serial run yields, that a slice imports none of the package’s own modules, and that a nested package is not scanned as part of the enclosing one.

4.3 Package-scale salvage

Table 3 reports `harvest` runs in which every “compiles” is an actual compiler invocation on the emitted slice, recorded in the run’s `manifest.json`.

A detail of the self-application is worth stating precisely, because the two counts in this paper differ by one: `blocks` finds 163 top-level declaration blocks in `leanmend`’s own package, of which 162 are named. The exception is an anonymous `instance`, and `harvest` addresses declarations by name, so it neither slices nor claims it; the 162 it does address all compile.

package	decls	slices that compile	note
<code>leanmend</code> itself (12 modules, 2,572 lines)	162	162	self-applied
<code>test/broken</code>	7	3	4 failures, broken on purpose

Table 3: Package-scale harvest, measured with the binary built from this source.

subsystem	files	lines	theorems
<code>RequestProject/Catalog</code> (verified catalog builder)	9	1,762	45
<code>RequestProject/ComplexityLattice</code>	7	1,211	78
<code>RequestProject/SelfHosting</code>	12	2,772	101
<code>RequestProject/Unified</code>	5	1,102	65
<code>RequestProject/ZkPoP</code>	15	4,201	140

Table 4: The verified library. `lake build` succeeds (8,092 jobs); there are no `sorry`s.

The interesting part is the *size* of what is recovered. Across the 162 slices of the self-application, the median slice contains 3 declarations and 63.5 lines, against a package of 162 declarations and 2,572 lines; the mean is 12.5 declarations, and the largest slice — for `main`, which legitimately reaches almost everything — is 159 declarations and 2,566 lines. In other words, for a typical declaration the tool identifies a self-contained, compilable neighbourhood roughly 2% of the package’s size. That is the difference between “the package is broken” and “here is a minimal working example of the part you asked about.”

For the broken fixture, the four unsalvaged declarations are exactly those whose own transitive dependencies contain the deliberate defects (a type error, an ill-formed statement, and two consumers of them), and each is reported with the compiler errors of its slice.

4.4 Threats to validity

The scanner validation checks structural properties against an independent implementation, not against Lean’s parser; a block model that both implementations get wrong in the same way would pass. Block detection is column-0 based, matching the Lean style guide, so an indented top-level command is absorbed into the previous block. Harvest measures salvage on two packages of this repository and one fixture; we do not claim a salvage rate for arbitrary projects, and the self-application is favourable in that the tool’s own package is Mathlib-free. The unused-import warning is a heuristic and can only ever be a warning (§3.4). Finally, “compiles” means the emitted slice elaborates in the package’s environment; it does not mean the slice is the *minimum* such slice unless `--minimize` was used.

5 The verified library

The repository also carries a Lean library — 55 files, 11,289 lines, 429 theorems, no `sorry` — in five subsystems (Table 4). It is not incidental to the tool: one subsystem takes the tool’s own harvest output as its object of study, and another replaces a Python generator the repository used to trust with a Lean program whose invariants are theorems.

5.1 A catalog builder whose invariants are theorems

`RequestProject/Catalog` scans Lean sources, builds a catalog of declarations with digests and dependency indices, computes coverage vectors, and emits the JSON reports and a generated Lean self-catalog. It is exposed as the executable `catalogtool` and it replaces a pair of Python

scripts; `catalogtool check` re-derives every generated artifact from the sources and exits non-zero on any disagreement (it exits 0 on this commit). The point of the rewrite is that its output can be reasoned about instead of trusted:

$$\text{checkSystem_buildCatalog} : \forall \text{srcs}, \text{checkSystem} (\text{buildCatalog srcs}) = \text{true}$$

— *every* catalog it builds satisfies the catalog invariant — together with `buildCatalog_acyclic` (every dependency index of an entry is strictly smaller than the entry’s own, so the emitted graph is acyclic), `buildCatalog_digest_inj` (distinct bounded entries receive distinct digests), `closure_depClosed` and four `coverage_*_le` bounds.

5.2 Self-hosting circuits

`RequestProject/SelfHosting` builds a finite arithmetic circuit language with an evaluator, a universal circuit `univ m n` that interprets an encoded circuit given as input, and unconditionally binding (non-compressing) digests with Merkle path soundness. The specification theorem is

$$\text{univ_spec} : (\text{univ } m \ n). \text{eval } (x \mathrel{++} \text{encodeGates } C.\text{gates} \mathrel{++} [o]) = [(C.\text{wires } x).\text{getD } o \ 0],$$

i.e. meta-evaluation of an encoded circuit agrees with direct evaluation; and `univ_selfhost_chain` iterates it one level up, running `univ` on the encoding of `univ` itself. `selfCircuit_selfDescribes` states that the universal circuit’s own description verifies and decodes back to itself. Recursion here reaches depth 2; there is no zero-knowledge or succinctness content in this subsystem, and the digests grow with their input.

5.3 The complexity lattice, applied to the tool

`RequestProject/ComplexityLattice` measures how much data an object can hold by anchoring it to reference structures of exactly known order — finite simple groups, their maximal subgroups, divisors of those orders, and members of infinite families with parameters fixed. Fitting is pigeonhole and is proved to be exactly that:

$$\text{fits_iff_embeds} : \text{Fits } X \ G \iff \text{Nonempty } (X \hookrightarrow \text{Fin } G.\text{order}).$$

Each register node records whether its order was *proved* inside Lean or *entered* as reference data, and the file checks the entered data for internal consistency.

The subsystem’s last module closes the loop with §3.3: the objects located are `leanmend`’s own slice spaces. A module with n top-level declarations has 2^n slices; the tool’s own package has 163 declaration blocks, so its harvest ranges over `SliceSpace 163`, and `harvest_capacity_vs_cert` states that the slice space has $\log_2$ -capacity 163 while the certificate that locates it in the lattice has capacity 8 — a 155-bit gap, stated as the exact identity $|\text{SliceCert}| \cdot 2^{155} = |\text{SliceSpace } 163|$. Where the numbers are unflattering they are reported: at alphabet size 64 the 33-bit row certificate is *larger* than the 30-bit object it certifies, and the intended divergence appears only at much larger scales.

5.4 Architecture, formalised

`RequestProject/Unified` formalises the joining material between the other subsystems: packaging as a contract (a client written against the interface keeps its guarantee under any realization, `hot_swap`; a behaviour-changing swap cannot be silent, `digest_detects_swap`), a Galois tower of shadows in which the adjunction inequalities are actually checked rather than assumed, a defect calculus with each form instantiated, and a self-hosting ladder discharged rung by rung. Its sharpest statement is a negative one: `acceptance_alone_insufficient`

exhibits a *sound* always-accepting recursion tower whose acceptance implies nothing about the property of interest — the formal reason a self-certifying tower needs a base case that stops the regress. The Garden-of-Eden module proves the finite Moore–Myhill equivalence (surjective $\iff$ pre-injective, by pigeonhole) and states explicitly that nothing is claimed about sofic groups.

5.5 zkPoP: arithmetize the checker, not the trace

`RequestProject/ZkPoP` is the largest subsystem: a typed object language whose *type checker* is arithmetized into field constraints, a Tseitin-style flattener from multivariate polynomial systems to flat gate circuits, a catalog of five specified gadgets, Pedersen commitments with Schnorr proofs of knowledge and an extractor, a self-hosting universal circuit with two-level recursion, and N -of- M shard distribution. The chain of headline theorems is:

- `Tm.checker_sound`: if the checker infers type τ for t , evaluation of t produces a value of type τ — the semantic bridge, proved first;
- `typeSat_iff`: the field-constraint system is satisfiable exactly when the checker accepts, so the arithmetization is faithful in both directions;
- `flatSys_sat_iff`: the flattened circuit is satisfiable by an assignment extending σ exactly when the high-level system is satisfied by σ ;
- `circuitRel_sound` / `coreCircuitRel_sound`: a satisfying assignment entails that the program really reduces to a value of the claimed type;
- `univ_selfhost`: the universal checker, handed its own flat circuit as private witness, accepts exactly the assignments that circuit accepts, at the exact hosting cost `entryHostSize( $n, \ell$ ) =  $\ell(3n + 8)$` ;
- `pedersen_binding_reduces_to_dlog`: a double opening *yields* the discrete logarithm, $g = \frac{r'-r}{m-m'} \cdot h$ — a reduction, proved, with hardness assumed nowhere; and `schnorr_special_soundness`, where two accepting transcripts with different challenges yield a genuine opening;
- `erasure_not_private`: a proved *negation* — plain systematic erasure fails the privacy audit — alongside `ramp_private` for the randomised encoding.

What is not claimed is stated as plainly as what is. Nothing here is a SNARK: there is no polynomial commitment scheme, no proof-size or verifier-time bound, and no trusted setup. Hiding and zero knowledge are perfect (information-theoretic) statements about the group, not computational-indistinguishability arguments. Tag distinctness uses `CharZero F` rather than a concrete prime field. The Merkle combiner is a fixed low-degree algebraic expression, not a collision-resistant hash, and no collision resistance is claimed. Universality is bounded to circuits of a fixed wire shape, and the object language is small — this is a worked instance of “arithmetize the checker, not the trace,” not a checker for Lean 4.

5.6 The axiom audit is part of the build

For each headline theorem cited above, the axiom closure is pinned inside the build. `RequestProject/ZkPoP/EndToEnd.lean` carries nine such blocks and `RequestProject/PaperAudit.lean` eight more, each of the form

```

/-- info: 'ZkPoP.end_to_end_recursive' depends on axioms: [propext, Classical.choice, Quot.
    sound] -/
#guard_msgs in
#print axioms ZkPoP.end_to_end_recursive

```

so the build *fails* if a cited theorem ever acquires a different axiom closure. All seventeen resolve to standard axioms only, and three do better: `Tm.checker_sound` needs `propext` alone, `Unified.acceptance_alone_insufficient` needs `propext` and `Quot.sound`, and `Unified.hot_swap` depends on no axioms at all. No project declaration is an axiom, no `@[implemented_by]` is used, and no theorem uses `native_decide`.

6 Introspection: measuring the repository rather than describing it

A repository that ships numbers about itself should generate them. `introspect/` is a 15-phase pipeline that runs the real builds and binaries and writes what it finds to `introspection/`, `STATUS.md` and the presentation material: build and test results with wall times and exit codes; every constant of the built environment with its type, term size, dependencies and `CollectAxioms` closure, dumped through a Lean module that loads the `.olean`s; profiled executions of the compiled binaries; spec items parsed from the READMEs and doc comments with coverage in both directions; graphs of imports, declarations, proofs, axioms and runtime; and a validator that fails if any recorded source, declaration, exemplar or figure does not resolve to a real artifact.

Two disciplines are worth naming. First, **evidence tiers are kept apart** on disk and in the report: measured facts (builds, tests, kernel terms, runtimes) never mix with exploratory lenses (PCA, clusters, prime fibers), and capacity bounds are reported as bounds with a stated justification, never as a score. Second, **every stochastic step records its seed**, so a re-run on the same commit reproduces the same clusters, regions and exemplars.

The weak link is stated rather than hidden: the pipeline’s Python is untrusted, and the correspondence between the Lean sources and the generated catalogs is not itself machine-checked. That is precisely the gap the Lean catalog builder (§5) was written to close for the catalog, and it remains open for the rest of the pipeline.

7 Limitations

Beyond the per-section caveats: `leanmend` performs *mechanical* repair only and will never attempt to prove anything, so a package whose only problem is a genuinely broken proof is left with an explicit `sorry`, not a fix. Slicing is name-level and textual, with the compiler as the arbiter; it does not understand elaboration-time effects, so an instance or notation dependency that no name records can make a slice fail (the failure is reported, never silently accepted). `kernel` requires a `lean4lean` binary built with the same toolchain as the package under test, since it reads that package’s `.olean` files. The verified library’s guarantees are about the objects it defines, not about Lean 4 itself; in particular the self-hosting and `zkPoP` subsystems are about their own small object languages.

8 Related work

Lean’s own `lake` provides building and dependency management but no partial-failure recovery. Mathlib’s tooling ecosystem includes import-graph analysis and shake-style unused-import detection, which `graph` resembles in intent but not in operating conditions: `leanmend` works

without building the package, which is the case that matters here. Delta debugging [2] is the standard technique for minimising failing inputs, and `--minimize` is a straightforward instance of it applied to declaration-level dependencies rather than characters. `lean4lean` [1] provides the independent kernel we integrate. The idea of extracting a self-contained minimal working example from a large development is folklore among proof engineers; the contribution here is doing it across module boundaries on a package that does not build, and at package scale.

9 Reproducing every number in this paper

```
# the verified library: 8092 jobs, no sorry
lake build && rg -n 'sorry|admit' -g '*.lean' RequestProject

# the axiom audit (fails the build if a closure changes)
lake build RequestProject.PaperAudit

# the tool, and its 69 end-to-end assertions
cd leanmend && lake build && bash test/run_tests.sh

# scanner validation: 200 Mathlib files, properties P1-P3
python3 test/scanner_validation.py 200

# package-scale salvage: 162/162 and 3/7
./lake/build/bin/leanmend harvest . --out /tmp/salvage --jobs 4
./lake/build/bin/leanmend harvest test/broken --out /tmp/salvage_broken

# the diagnostics of Listings 1 and 2
./lake/build/bin/leanmend graph test/graphpkg
./lake/build/bin/leanmend doctor test/broken
```

Environment: Lean 4.28.0, Lake 5.0.0, Mathlib at commit 8f9d9cff6bd728b17a24e163c9402775d9e6a365. `leanmend` itself has no dependencies.

10 Conclusion

The request that started this repository was to take a Lean project apart and get the parts that still work to build. `leanmend` does that: it diagnoses a package without being able to build it, tells the user which *declarations* are broken rather than which lines, repairs the mechanical breakage without inventing mathematics, and recovers a compilable, self-contained slice for each declaration — all 162 named declarations of its own package, with a median slice of 3 declarations. Around it sits a **sorry-free** library whose headline theorems’ axiom closures are checked by the build itself, and one of whose subsystems takes the tool’s own output as its object of study. What the artifacts do not establish is written down next to what they do.

References

- [1] M. Carneiro. *lean4lean*: an implementation of the Lean 4 kernel in Lean 4. <https://github.com/digama0/lean4lean>.
- [2] A. Zeller and R. Hildebrandt. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering* 28(2), 2002.